Institut National des Sciences Appliquées et de Technologie

LittleBig Connection

Université de Carthage

GRADUATION PROJECT · END-OF-STUDIES INTERNSHIP DISSERTATION

Design, Implementation, and Pipeline Integration of a Smoke Test Suite

as a CI/CD Quality Gate for a Multi-Universe SaaS Platform

Carried out at **LittleBig Connection · Mantu Group**

Presented by **Ahmed SILINI**

ACADEMIC SUPERVISOR
Dr. Sana HAMDI

COMPANY SUPERVISOR
Mrs. Hajer AKAICHI

COMPANY MENTOR
Mr. Zied KLOUZ

Academic Year 2025 – 2026

LittleBig Connection

Table of contents

LittleBig Connection

Table of contents

01 **Context & Project Presentation**
LittleBig Connection · the problem · the proposed gate · where smoke testing fits

LittleBig Connection

Table of contents

01 **Context & Project Presentation**
LittleBig Connection · the problem · the proposed gate · where smoke testing fits

02 **Requirements & Architecture**
what the suite must guarantee · a four-layer framework

LittleBig Connection

Table of contents

01 **Context & Project Presentation**
LittleBig Connection · the problem · the proposed gate · where smoke testing fits

02 **Requirements & Architecture**
what the suite must guarantee · a four-layer framework

03 **Implementation**
organised stack · auth fast path · code quality · CI/CD gate · Teams card · dashboard

LittleBig Connection

Table of contents

01 **Context & Project Presentation**
LittleBig Connection · the problem · the proposed gate · where smoke testing fits

02 **Requirements & Architecture**
what the suite must guarantee · a four-layer framework

03 **Implementation**
organised stack · auth fast path · code quality · CI/CD gate · Teams card · dashboard

04 **Results & Evaluation**
four formula-based metrics · measured on real pipeline runs · all targets met

LittleBig Connection

Table of contents

01 **Context & Project Presentation**
LittleBig Connection · the problem · the proposed gate · where smoke testing fits

02 **Requirements & Architecture**
what the suite must guarantee · a four-layer framework

03 **Implementation**
organised stack · auth fast path · code quality · CI/CD gate · Teams card · dashboard

04 **Results & Evaluation**
four formula-based metrics · measured on real pipeline runs · all targets met

05 **Conclusion & Perspectives**
what was delivered · honest limitations · six concrete next steps

Plus a ≈ 5-minute live demo · 18 smoke tests · 20 priority-one critical journeys · 4 domains · ~2 min parallel runtime

LittleBig Connection

01

Context & Project Presentation

LittleBig Connection · the problem · the proposed gate · where smoke testing fits

Mantu Group & LittleBig Connection

LittleBig Connection

Founded 2013

→

mantu

Joined 2015

a global independent consulting group



A single SaaS platform replacing separate procurement tools : the system this smoke suite protects.

Mantu Group & LittleBig Connection

LittleBig Connection — Founded 2013 → mantu — Joined 2015

a global independent consulting group



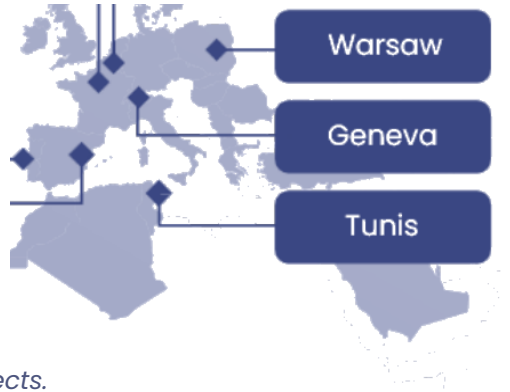
Two products, one platform : a Marketplace to find and contract external talent, and Eleven VMS to manage every provider relationship end to end.

Connect
The Marketplace

Find, source and contract external talent.

Eleven VMS
Vendor **M**anagement **S**ystem

Manage the whole provider relationship.



A single SaaS platform replacing separate procurement tools : the system this smoke suite protects.

Mantu Group & LittleBig Connection

50+
Countries in the world

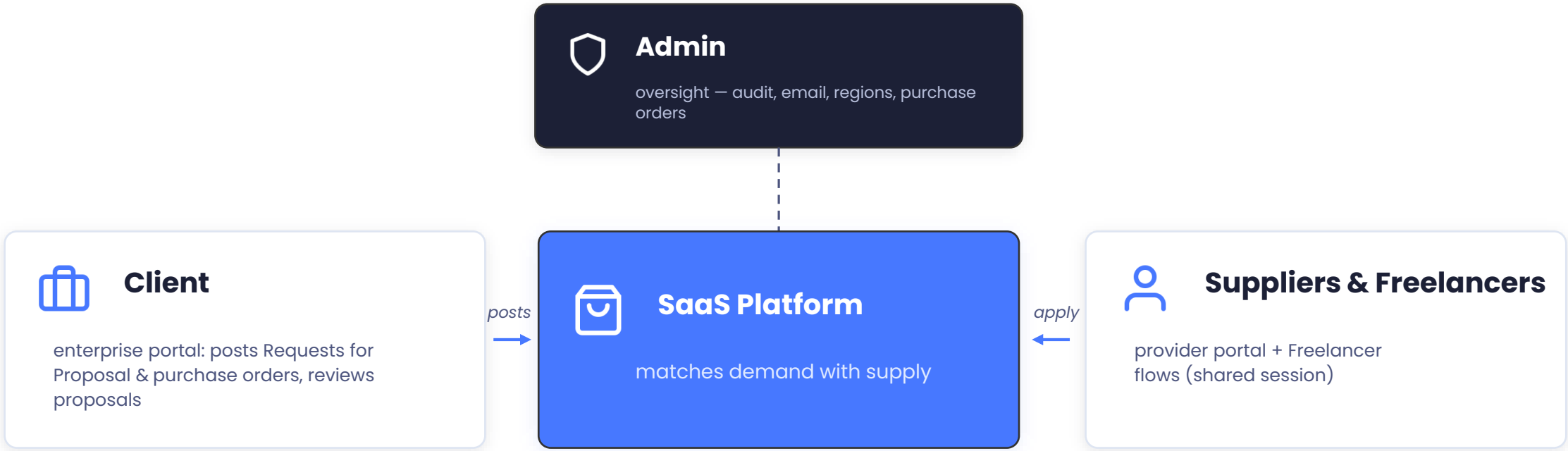


Mantu Group & LittleBig Connection

~400
Employees



Project Context — Platform Universes



Freelancer flows are folded under Supplier — same session state, kept explicit in reports by a **Supplier Freelancer** test prefix.

Four universes → four test domains: Auth, Admin, Client, Supplier.

The Problem

> **Nothing sucks the blood of Continuous Integration more than a build that takes a long time.**

Martin Fowler

The Problem

> **Nothing sucks the blood of Continuous Integration more than a build that takes a long time.**
>
> *Martin Fowler*

Our pipeline took eight hours to answer one question — and even then, the answer couldn't be trusted.

8+ hours

to catch a broken deploy — the only signal we had

Flakiness: unknown
no visibility on false failures at all

Tests tightly coupled
could not run in isolation

No deploy gate
no fast health check per change

A green pipeline was no longer a trustworthy signal of production health.

The Problem

> **Nothing sucks the blood of Continuous Integration more than a build that takes a long time.**
>
> *Martin Fowler*

Our pipeline took eight hours to answer one question — and even then, the answer couldn't be trusted.

1. **Developers wait**
2. **Releases wait**
3. **Stakeholders wait**

The speed of this one gate set the speed of everything behind it.

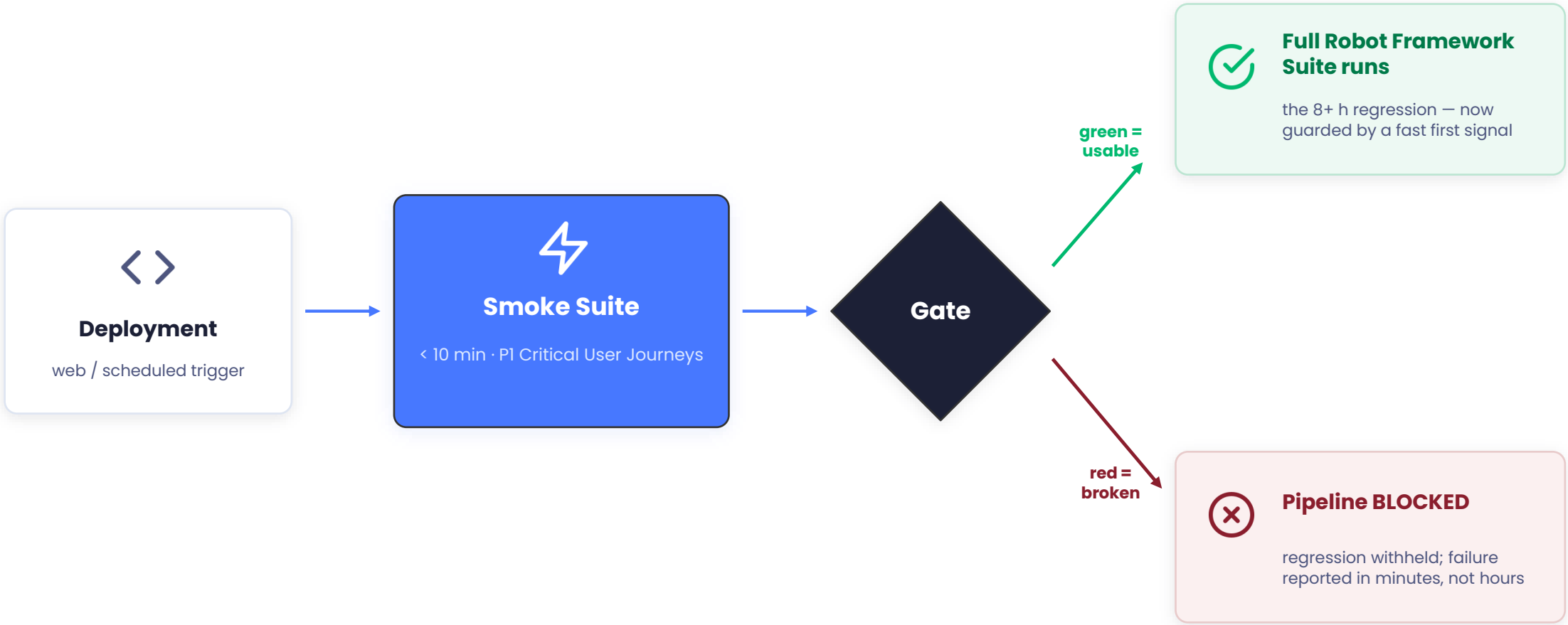
The Proposed Solution

Smoke Suite (Robot Framework + Browser Library)

Same syntax as the regression suite

18 global-path checks

The Proposed Solution



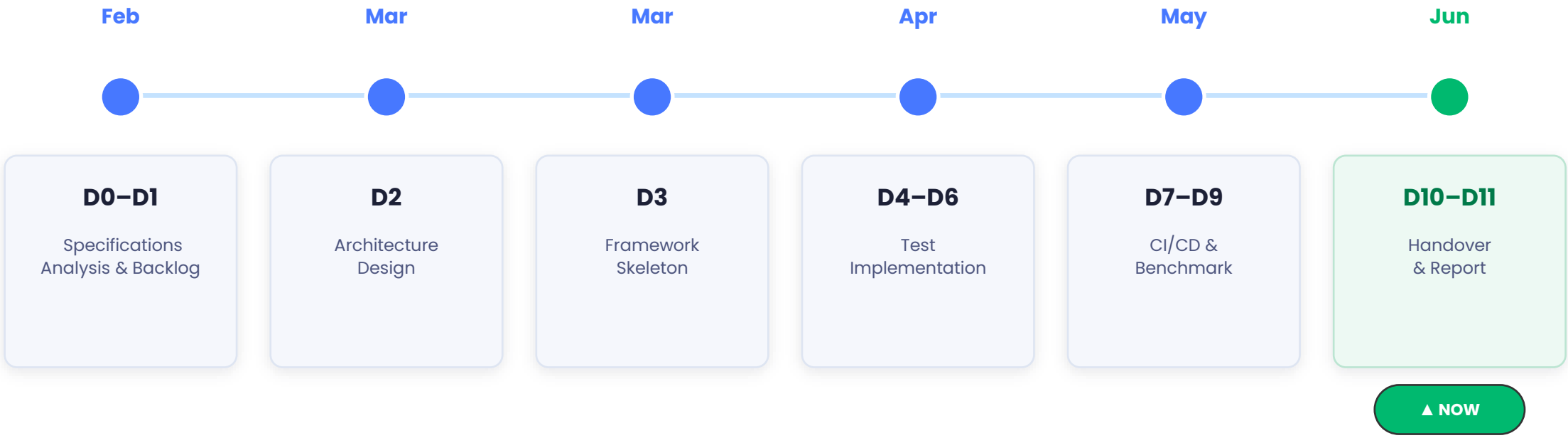
Smoke Suite (Robot Framework + Browser Library)

Same syntax as the regression suite

18 global-path checks

Methodology & Timeline

Iterative & Incremental lifecycle · Kanban on Jira as the within-phase workflow · 6 phases · 11 deliverables



Each phase ships a validated deliverable the next one builds upon — feedback absorbed without ceremony overhead.

Inside Each Phase

one turn of the iterative loop per phase · validated at a phase review · day-to-day flow tracked on a Jira Kanban board

Inside Each Phase

one turn of the iterative loop per phase · validated at a phase review · day-to-day flow tracked on a Jira Kanban board

ITERATIVE & INCREMENTAL — THE LOOP

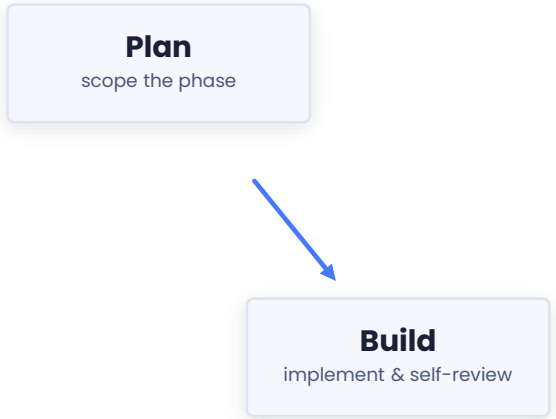
Plan
scope the phase

The same loop ran six times — one board carried both the phase deliverables and the day-to-day task flow.

Inside Each Phase

one turn of the iterative loop per phase · validated at a phase review · day-to-day flow tracked on a Jira Kanban board

ITERATIVE & INCREMENTAL — THE LOOP

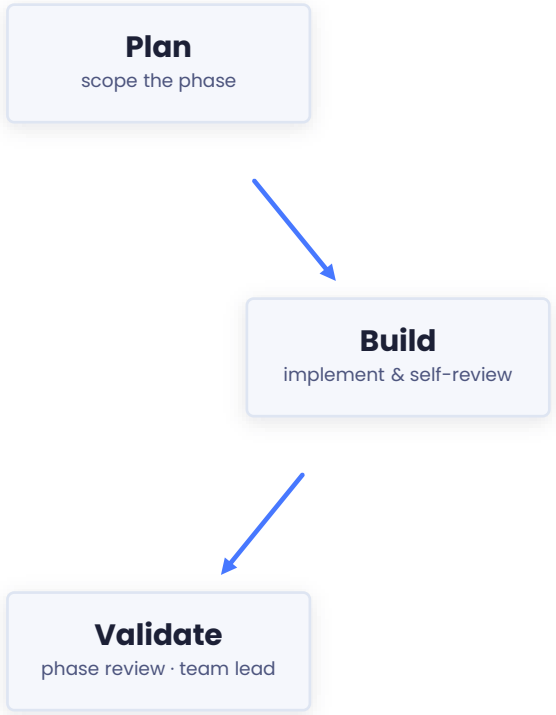


The same loop ran six times — one board carried both the phase deliverables and the day-to-day task flow.

Inside Each Phase

one turn of the iterative loop per phase · validated at a phase review · day-to-day flow tracked on a Jira Kanban board

ITERATIVE & INCREMENTAL — THE LOOP

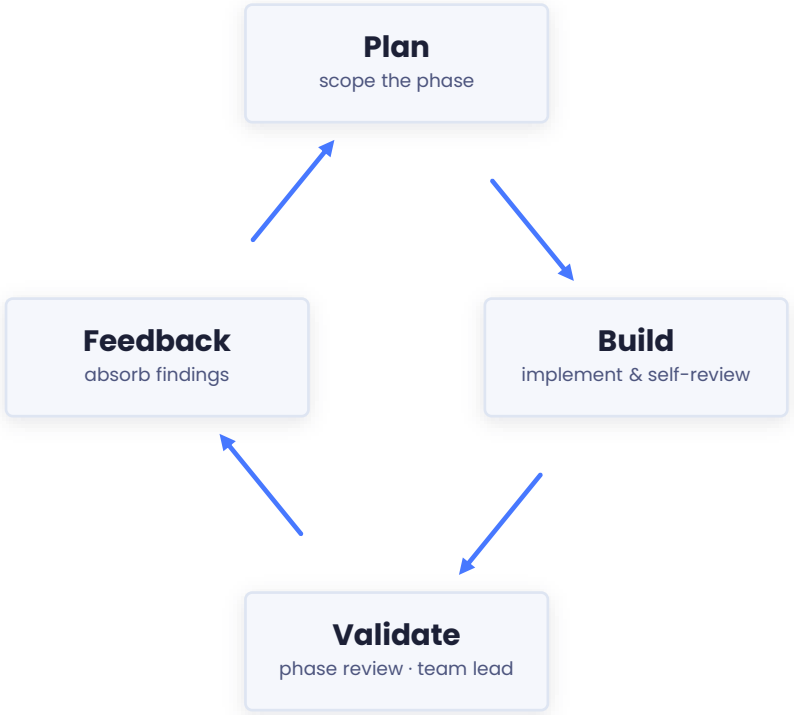


The same loop ran six times — one board carried both the phase deliverables and the day-to-day task flow.

Inside Each Phase

one turn of the iterative loop per phase · validated at a phase review · day-to-day flow tracked on a Jira Kanban board

ITERATIVE & INCREMENTAL — THE LOOP

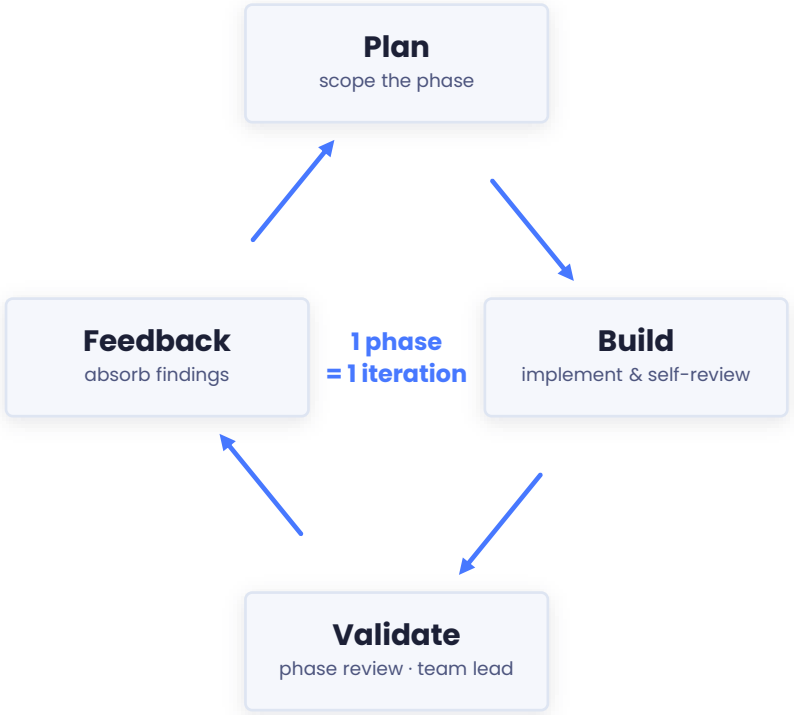


The same loop ran six times — one board carried both the phase deliverables and the day-to-day task flow.

Inside Each Phase

one turn of the iterative loop per phase · validated at a phase review · day-to-day flow tracked on a Jira Kanban board

ITERATIVE & INCREMENTAL — THE LOOP



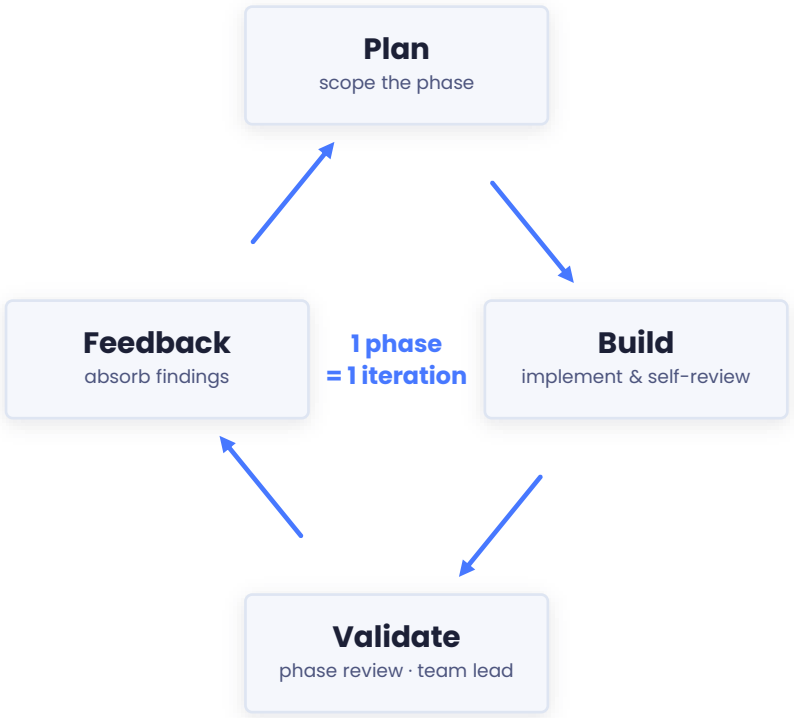
each turn ships a validated deliverable across six phases

The same loop ran six times — one board carried both the phase deliverables and the day-to-day task flow.

Inside Each Phase

one turn of the iterative loop per phase · validated at a phase review · day-to-day flow tracked on a Jira Kanban board

ITERATIVE & INCREMENTAL — THE LOOP



each turn ships a validated deliverable across six phases

KANBAN ON JIRA — THE WITHIN-PHASE WORKFLOW

TO DO 3	IN PROGRESS 2	IN REVIEW 2	DONE 34
ADMIN Region config smoke	SUPPLIER Activity Report list smoke	ADMIN Audit log smoke	AUTH Fast-path session caching
CLIENT Partner-domain smoke	CLIENT Draft Request for Proposal create + delete	DOCS D5 — Supplier deliverable	CLIENT Request for Proposal list & detail smoke
CI/CD Weekend metrics probe			**+ 31 more** *across all phases*

representative mid-project board state (report Fig. 1.6) — tasks tagged by universe & category

The same loop ran six times — one board carried both the phase deliverables and the day-to-day task flow.

LittleBig Connection

02

Requirements & Architecture

what the suite must guarantee · 18 tests across 20 priority-one journeys · a four-layer framework

Requirements at a Glance

Functional Requirements

FR-01/02/03	Automate smoke tests for all 4 universes
FR-04	Each test fully independent — owns its data
FR-05	Auth fast path for all 4 user roles
FR-06	Multi-environment via configuration only
FR-07	Dedicated CI/CD gate stage in the pipeline
FR-08/09	Reports · metrics dashboard · Teams alerts

Non-Functional Requirements

NFR-01	Performance	< 10 min
NFR-02	Reliability	< 2% flakiness
NFR-03	Maintainability	Page Object Model
NFR-04	Scalability	add tests, change nothing
NFR-05	Portability	config-only env switch

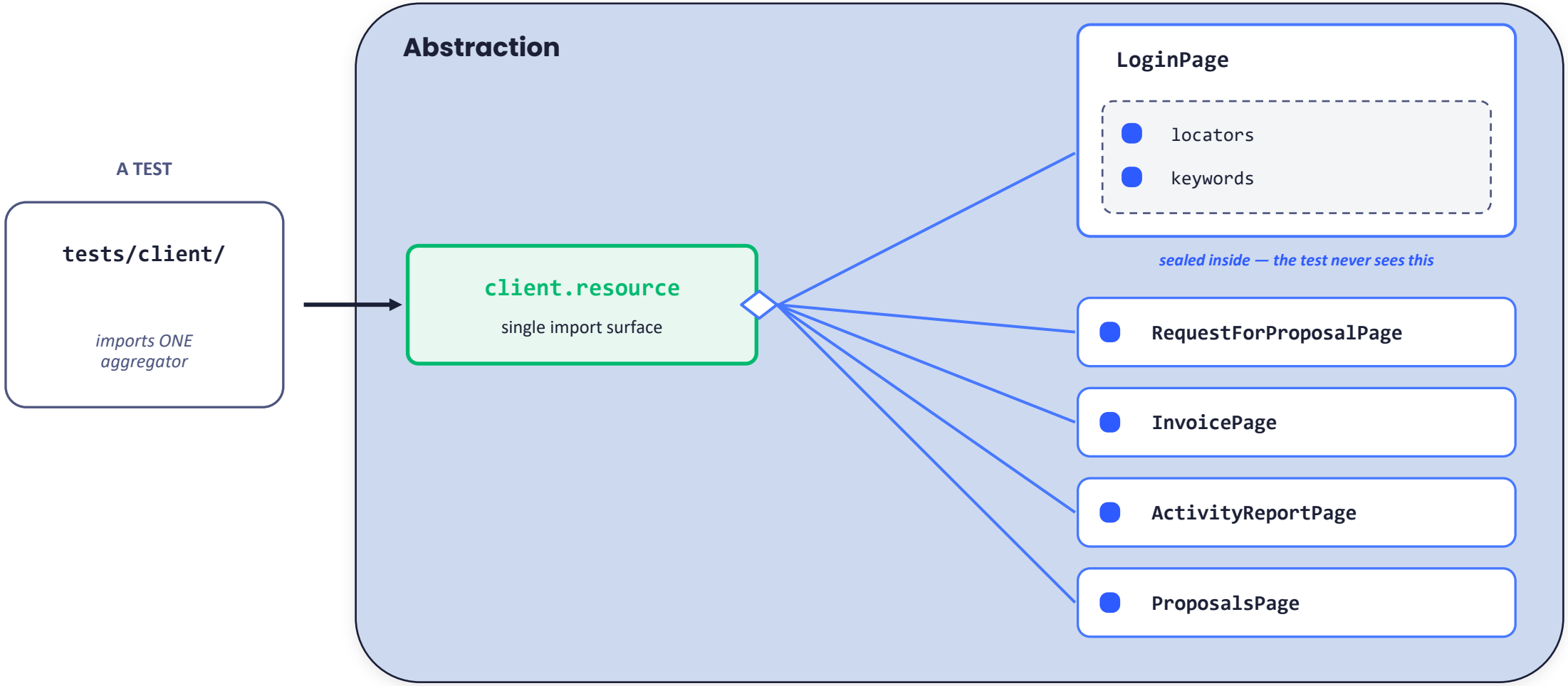
4-Layer Framework Architecture

4	**Reporting & CI/CD**	gate logic · HTML report + video + HAR · Teams Adaptive Card · GitLab Pages dashboard
3	**Test Execution**	18 smoke tests · 20 P1 journeys · one suite per functional area per universe
2	**Abstraction**	Robot Framework resource per page · universe Facade aggregators · AgGrid + PrimeVue table utilities
1	**Infrastructure**	Docker container · Playwright / Chromium engine · GitLab runner

Two design patterns govern the object model: **Page Object Model** (one resource per page) and **Facade** (one aggregator per universe). Each layer depends only on those beneath it.

Design Patterns — Page Object Model & Facade

The same two patterns repeat identically across all four universes.



Facade gives one way in • **Page Object Model** seals locators away → **a UI change touches one file.**

Repository Layout

```
sanity-suite/
├─ infra/              # Dockerfile · run scripts · probe
├─ traceability/       # metrics + coverage + Teams
│    ├─ publish_metrics.py
│    ├─ analyze_probe.py
│    ├─ compute_coverage.py
│    └─ cuj_catalogue.yaml
├─ dashboard/          # HTML/JS/CSS + history.json
├─ utils/              # env · common · aggrid · table
├─ pages/              # page objects by universe
│    ├─ shared/   client/
│    └─ supplier/ admin/
├─ tests/              # suites mirror pages/ 1-to-1
│    └─ auth/ client/ supplier/ admin/
└─ .gitlab-ci.yml      # pipeline definition
```

pages/ + tests/

mirror one-to-one — a test suite per page object, organised by universe

traceability/

Python scripts compute metrics & coverage and post the Teams card

dashboard/

the GitLab Pages site + its append-only history.json data store

.gitlab-ci.yml

the 5-stage pipeline that wires everything together

LittleBig Connection

03

Implementation

Stack · auth fast path · test data · code quality · CI/CD gate · Teams card · metrics dashboard

Technology Stack

Test Authoring & Execution

- **Robot Framework** — test authoring + runner
- **Browser Library** — robotframework-browser
- **pabot** — parallel executor

Browser Engine & Runtime

- **Playwright** — engine via gRPC / Chrome DevTools Protocol
- **Chromium** — target browser
- **Docker** — Py 3.11 + Node + Chromium

CI/CD & Code Quality

- **GitLab CI/CD** — 5 stages + pages:deploy
- **Robocop** — Robot Framework linting on every merge request
- **Python** — metrics & traceability scripts

Reporting & Artifacts

- **HTML report · video · HAR · JUnit XML** — produced on every run
- **GitLab Pages** — static metrics dashboard over time

Notifications & Hand-off

- **Teams Webhook** — Adaptive Card alert per run
- **trigger-e2e** — fire-and-forget downstream regression

Robot Framework leads the stack — Playwright is the engine underneath, never the framing.

Robot Framework + Browser Library?

Robot Framework leads the stack — Playwright is the engine underneath, never the framing.

THE FOUR DECIDING CONSTRAINTS

1. **Parallel Multi-Context**
   rules out Cypress
2. **Chrome DevTools Protocol Execution**
   rules out Selenium & Robot Framework + SeleniumLibrary
3. **Native Video + HAR Capture**
   leaves only Playwright and Robot Framework + Browser Library
4. **Homogeneity with the Robot Framework suite**
   decides between the two

THE VERDICT

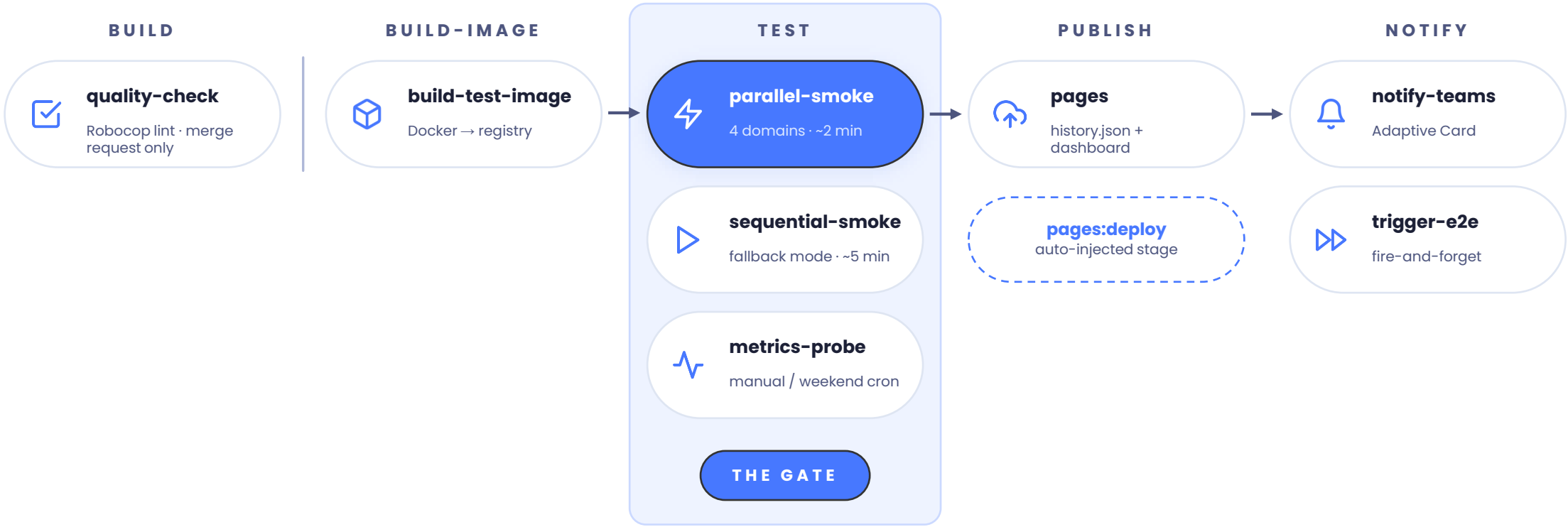
Robot Framework + Browser Library

the only stack that satisfies all four

- ✓ Tabular Robot Framework syntax → readable by non-programmers, same as the regression suite
- ✓ Playwright underneath via gRPC → Chrome DevTools Protocol
- ✓ No second language, no second suite to maintain

The CI/CD Pipeline

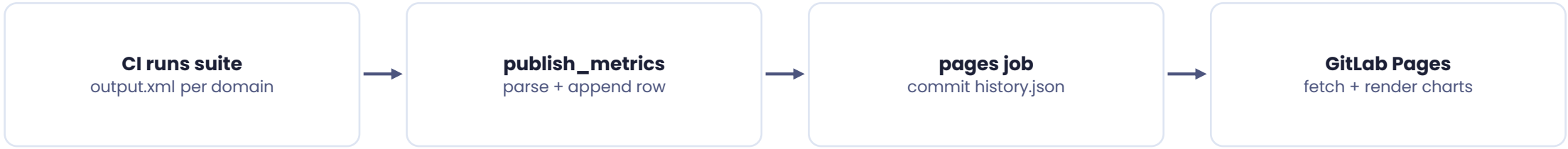
Triggers: web & scheduled runs only — merge request pushes start a separate lint-only pipeline.



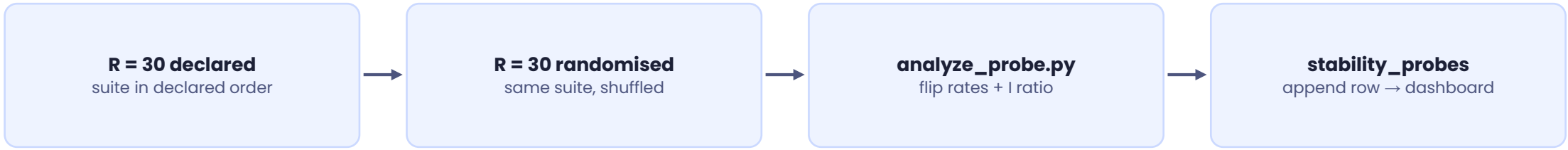
RERUN_ATTEMPTS retries failed tests (the merged result drives the gate) · BASE_URL routes to the right environment & credential family.

Data Flow & Stability Probe

Normal pipeline run



Stability probe — feeds flakiness φ and independence I



Probes launch from a manual play button or 2 weekend schedules → 6 probes per weekend, avoiding busy CI windows.

LittleBig Connection

04

Results & Evaluation

four formula-based metrics · measured on real pipeline runs · all targets met

System Health Check

THE FULL SUITE

~1,000

tests

*every detail checked · thorough, but takes **hours** to run*



THE SMOKE GATE

18

tests

*only what proves the platform is usable · about two **minutes***

Parallel: μ = 113 s ± 44 s (13 runs) · Sequential: μ = 292 s ± 87 s · both well below the ten-minute target.

Execution Time — 2.9× Faster than Selenium

Domain	Selenium	Robot Framework + Browser Library	Speed-up
Admin	1:35	0:41	**2.3×**
Client	6:13	1:58	**3.2×**
Supplier	6:23	1:47	**3.6×**
Total (17 matched tests)	**14:11**	**4:52**	**2.9×**

Largest gains on AgGrid-driven flows (Request for Proposal, Purchase Order, Activity Report) — native auto-wait replaces polling loops.

Flakiness & Independence — Evolution

13 stability probes · R = 30 declared + 30 randomised runs each

Apr 26

φ = **0.083**

flakiness rate

I = **0.810**

independence rate

✗ not yet

May 2

φ = **0.018**

flakiness rate

I = **0.920**

independence rate

✗ not yet

May 24

φ = **0.0077**

flakiness rate

I = **1.004**

independence rate

✓ BOTH PASS

φ declined monotonically over 13 probes — worst single test φ = 0.069 (a grid-heavy Supplier Purchase Order view).

All Quality Targets Met

Execution Time

target < 10 min

~2 min parallel
~5 min sequential

Flakiness Rate

target < 2% (0.02)

φ = 0.0077
(0.77%)

Test Independence

target I ≥ 1.0

I = 1.004
ordering agnostic

Critical Journey Coverage

target 100%

20 / 20 P1 CUJS
100%

Four for four — and the remaining functional and non-functional requirements are verified with concrete evidence in the report (Table 6.5).

LittleBig Connection



Live Demo

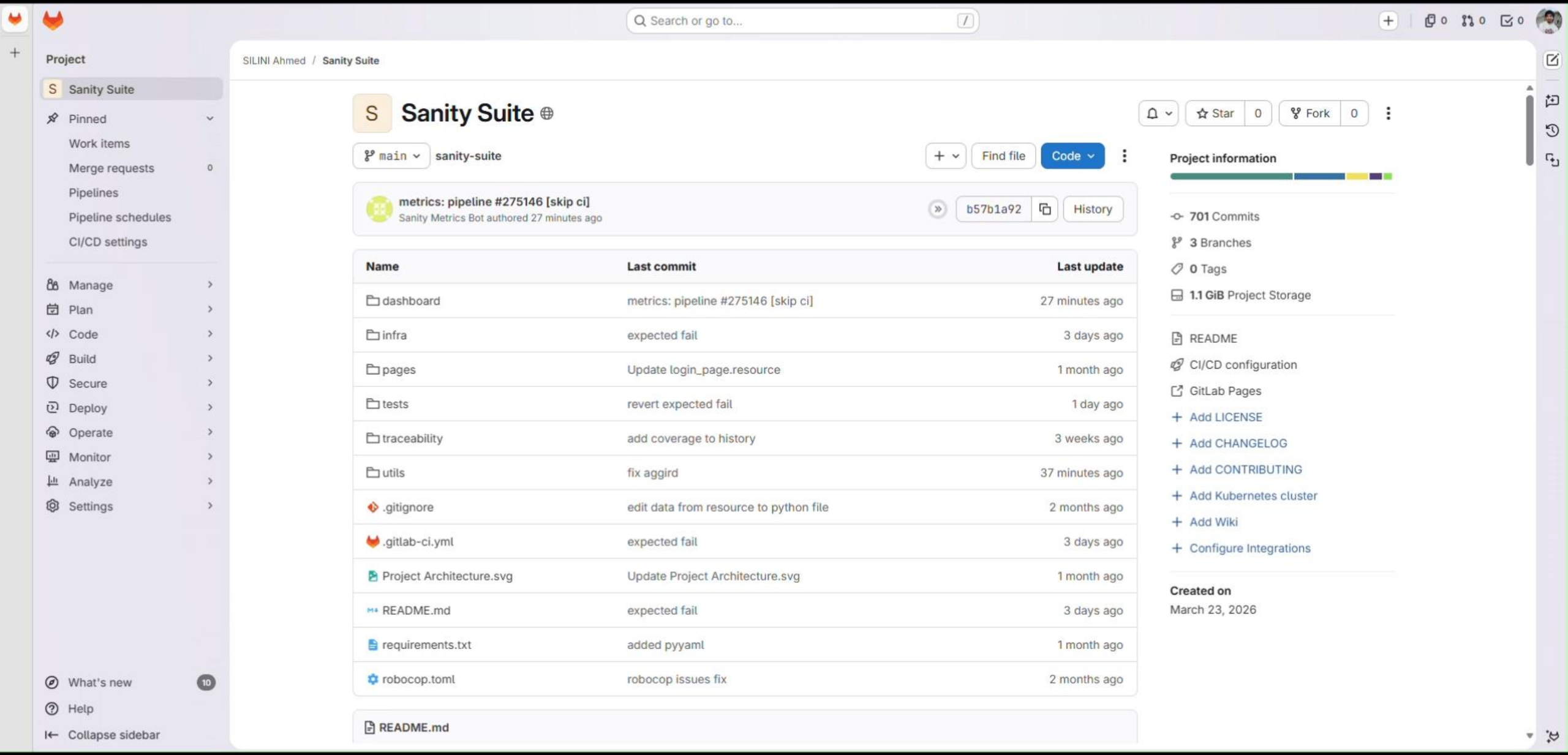
the gate and its instruments, on the real pipeline

1 · Trigger a pipeline

2 · Open the Teams card

3 · Watch the smoke gate

4 · Tour the dashboard


Search or go to...
Project
Sanity Suite
Pinned
Work items
Merge requests 0
Pipelines
Pipeline schedules
CI/CD settings
Manage
Plan
Code
Build
Secure
Deploy
Operate
Monitor
Analyze
Settings
What's new 10
Help
Collapse sidebar
SILINI Ahmed / Sanity Suite
Sanity Suite
Star 0
Fork 0
main
sanity-suite
Find file
Code
metrics: pipeline #275146 [skip ci]
Sanity Metrics Bot authored 27 minutes ago
b57b1a92
History
Name
Last commit
Last update
dashboard
metrics: pipeline #275146 [skip ci]
27 minutes ago
infra
expected fail
3 days ago
pages
Update login_page.resource
1 month ago
tests
revert expected fail
1 day ago
traceability
add coverage to history
3 weeks ago
utils
fix aggird
37 minutes ago
.gitignore
edit data from resource to python file
2 months ago
.gitlab-ci.yml
expected fail
3 days ago
Project Architecture.svg
Update Project Architecture.svg
1 month ago
README.md
expected fail
3 days ago
requirements.txt
added pyyaml
1 month ago
robocop.toml
robocop issues fix
2 months ago
README.md
Project information
701 Commits
3 Branches
0 Tags
1.1 GiB Project Storage
README
CI/CD configuration
GitLab Pages
Add LICENSE
Add CHANGELOG
Add CONTRIBUTING
Add Kubernetes cluster
Add Wiki
Configure Integrations
Created on
March 23, 2026

LittleBig Connection

05

Conclusion & Perspectives

what was delivered · honest limitations · six concrete next steps

Closing the Loop

> **Continuous Integration doesn't get rid of bugs, but it does make them dramatically easier to find.**

Martin Fowler

Closing the Loop

> **Continuous Integration doesn’t get rid of bugs, but it does make them dramatically easier to find.**

Martin Fowler

From an 8-hour wait to a 2-minute answer.

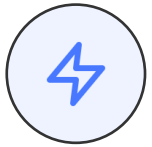
The blood a slow build was draining is back, **fast, honest feedback** on every change.

What Was Delivered



Layered Framework

18 smoke tests · Page Object Model + Facade · Robot Framework + Browser Library (Playwright engine)



Fast CI/CD Gate

~2 min parallel · blocks the 8 h regression on critical-path failure · two execution modes



Metrics Dashboard

GitLab Pages · 4 metrics tracked over time · LittleBig brand · append-only history.json per run



Teams Notification

Adaptive Card · pass / fail / partial states · collapsible failed tests · fire-and-forget trigger

Limitations & Threats to Validity



Scope of the gate

- Deferred priority-two/priority-three entries.
- Non-functional properties stay outside.
- A slow-but-correct page still passes.



Operational fragility

- Parallel mode saturates the shared runner pool.
- Shared pre-production accounts can break read-only tests.



No framework self-testing

- Page-object resources and utility keywords have no unit tests.
- Robocop catches structural drift, not semantic bugs.

Stated openly in the report (§6.5) — each result must be read against these boundaries; the perspectives address them directly.

Perspectives



NEAR-TERM

Consolidate

1 Close P2/P3 Critical User Journeys

Request for Proposal search and user management; the natural first additions once flows stabilise.

2 Xray Traceability

Feed xunit.xml to Jira Xray via REST API; the pipeline already produces it.

MID-TERM

Deepen

3 Cross-Universe Scenarios

Client action and Supplier reaction in one test: two browser contexts, one process.

4 Drift Alerting

Auto-alert when $\varphi > 2\%$ or $I < 1.0$, turning the dashboard from passive to active.

LONGER-TERM

Scale

5 Regression Migration

Squad-by-squad engine swap from Selenium to Browser Library, using this project as the pattern.

6 Intra-Domain Parallelism

Trim wall-clock time further as the suite grows beyond domain-level parallelism.

Thank you for your attention

Questions & Discussion

If you have any questions, please hesitate to ask ! :D

Institut National des Sciences Appliquées et de Technologie

LittleBig Connection

Université de Carthage

GRADUATION PROJECT · END-OF-STUDIES INTERNSHIP DISSERTATION

Design, Implementation, and Pipeline Integration of a Smoke Test Suite

as a CI/CD Quality Gate for a Multi-Universe SaaS Platform

Carried out at **LittleBig Connection · Mantu Group**

Presented by **Ahmed SILINI**

ACADEMIC SUPERVISOR
Mrs. Sana HAMDI

COMPANY SUPERVISOR
Mrs. Hajer AKAICHI

COMPANY MENTOR
Mr. Zied KLOUZ

Academic Year 2025 – 2026

LittleBig Connection

A

Appendices

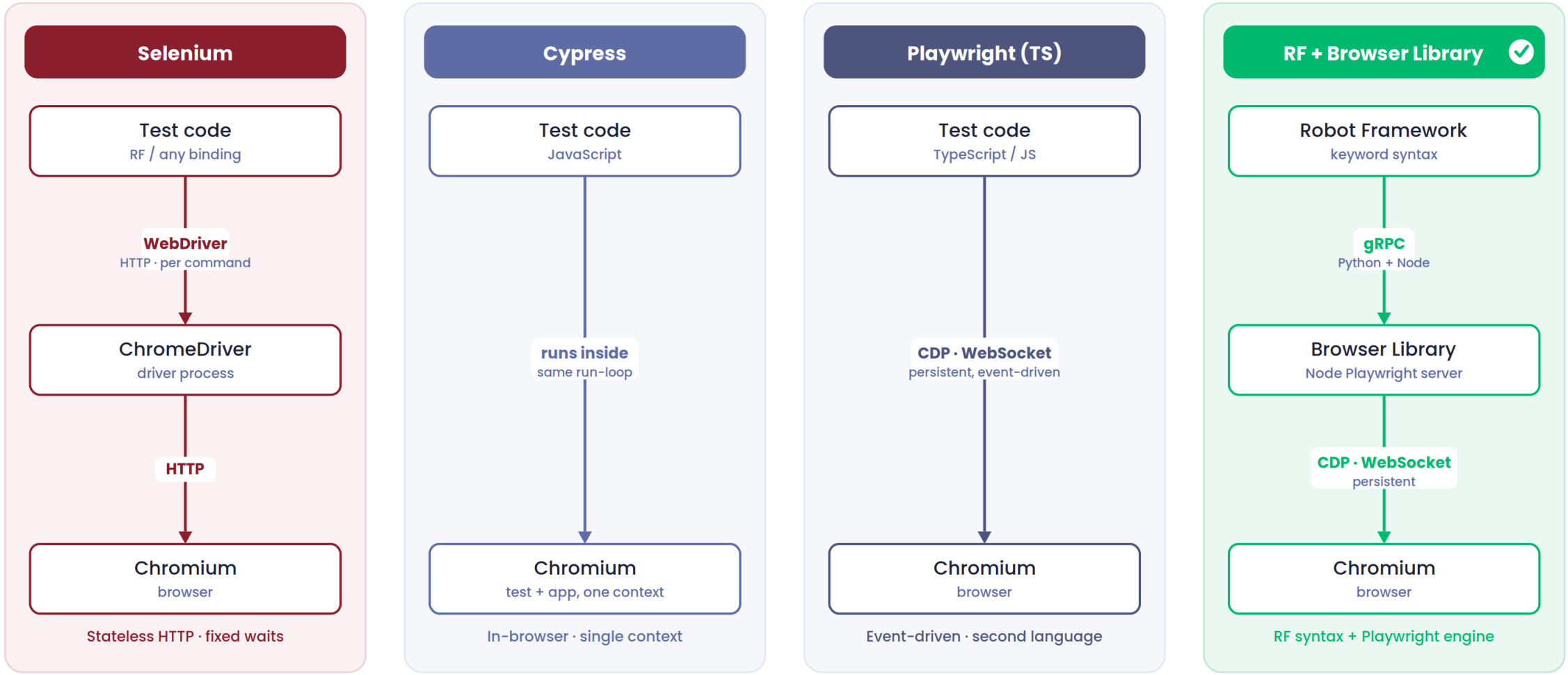
Reference material for the question period · architectures, comparisons, and metrics

Browser Automation Stacks Compared

Capability / Constraint	Selenium	Cypress	Playwright (TS)	RF + Browser Library ✓
Execution channel	HTTP per command	in-browser loop	WebSocket · CDP	WebSocket · CDP
Parallel multi-context	✗	✗ single ctx	✓	✓
Auto-wait on element state	✗ fixed waits	✓	✓	✓
Native video + HAR	✗	partial	✓	✓
Authoring language	many (incl. RF)	JavaScript	TypeScript / JS	RF keywords
Same syntax as LBC regression	✓ w/ Selenium	✗	✗	✓
Verdict	flaky by protocol	no cross-universe	second language	CHOSEN

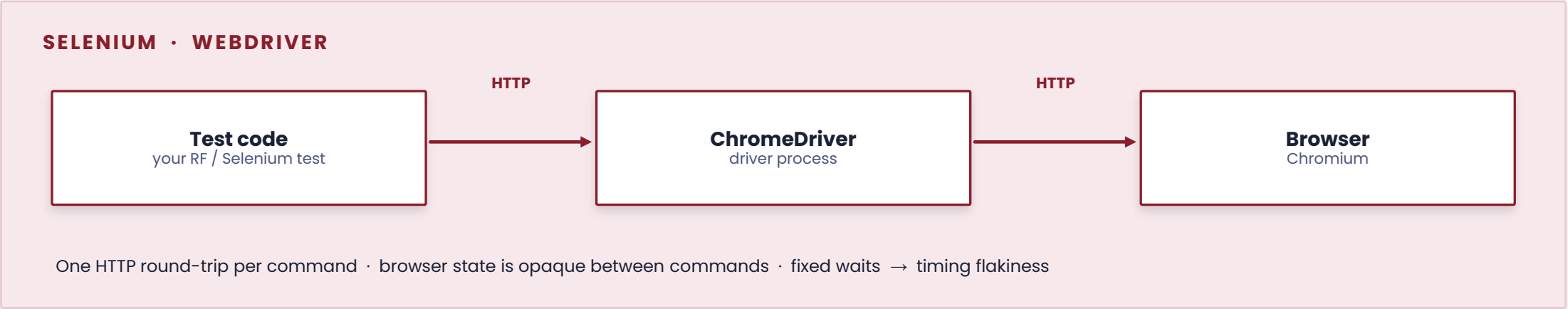
Capability rows are from each tool's official documentation; the speed figures are this project's own measurements (Chapter 6).

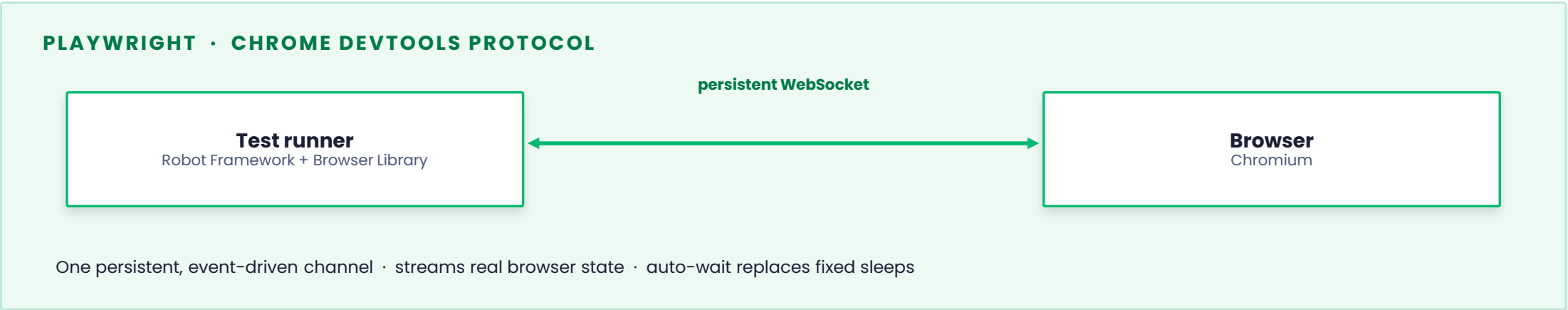
Browser Automation Stacks Compared



Every stack ends at the same browser; what differs is the language on top and the protocol in between.

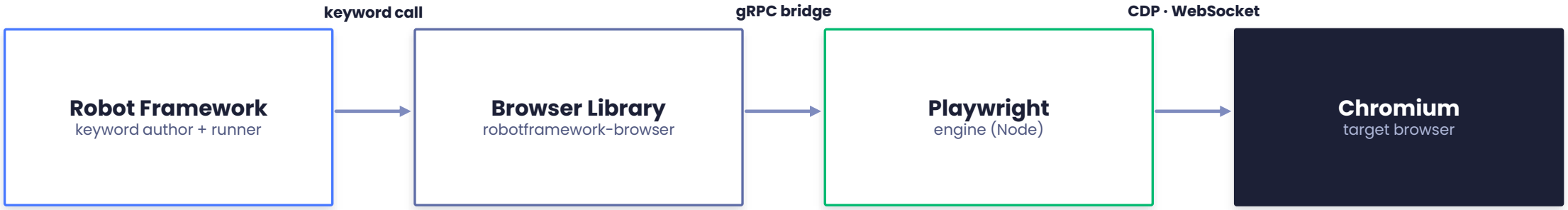
WebDriver vs. CDP — Why Playwright Is Less Flaky





The persistent, event-driven channel removes the per-command round-trip and lets the runner auto-wait on real element state.

The Stack — One Framework, One Engine



FRAMEWORK
Robot Framework
tabular keyword syntax — same as the regression suite

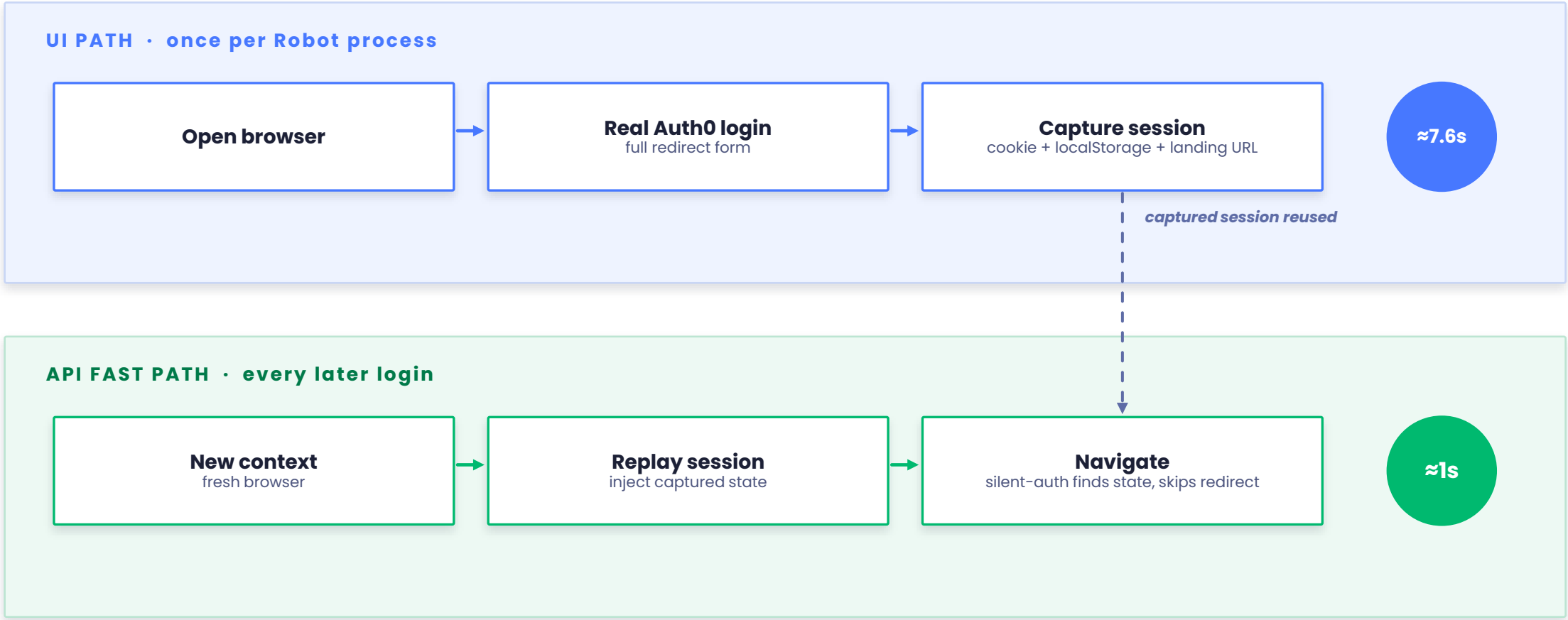
BINDING
Browser Library
exposes Playwright through Robot Framework keywords

ENGINE
Playwright
drives Chromium over the DevTools Protocol

The author writes the same keywords throughout; only the engine beneath the binding changed.

Robot Framework is the framework; Browser Library is the binding; Playwright is the engine underneath, reached over gRPC. Never a "Playwright project."

The Authentication Fast Path — ≈7.6 s to ≈1 s



Login itself is tested for real once per process; every later test reuses a genuine session as a precondition. If silent-auth changes, the keyword falls back to the UI path.

Requirements Verification — all 14 met

Req	What it demands	How it was verified / evidence
FR-01	Automate Client + Supplier smoke	Coverage metric — 6/6 Client, 7/7 Supplier (100%)
FR-02	Automate Admin + 2 Freelancer	Coverage metric — 4/4 Admin, 2 Freelancer in Supplier (100%)
FR-03	Each test fully independent	Independence ratio I = 1.004 (order-agnostic)
FR-04	Auth + session for all 4 roles	UI + API fast path live; every test runs unmodified
FR-05	Multi-environment by config	env.resource routes from BASE_URL
FR-06	Dedicated CI/CD gate stage	Gate observed blocking regression; first stage
FR-07	Test results report each run	HTML report + video + HAR + JUnit XML
FR-08	Metrics dashboard over time	GitLab Pages, 4 metrics, append-only history.json
FR-09	Notify team + monitor health	Teams Adaptive Card — pass / fail / partial
NFR-01	Performance < 10 min	≈113 s parallel / ≈292 s sequential
NFR-02	Flakiness < 2%	φ = 0.0077 (0.77%)
NFR-03	Maintainability via POM	Robocop on every MR; no selector crossed the boundary
NFR-04	Scalability — add tests freely	Shown in D6: one CUJ reformulated, one file changed
NFR-05	Portability — config-only switch	BASE_URL

Specified and designed in Chapter 3; verified with concrete evidence in Chapter 6. The four formula-based metrics verify the core requirements, the rest by evidence.

18 Tests across 20 Priority-One Journeys

18 smoke tests	20 P1 journeys	4 domains	≈2 min parallel runtime

Auth — 1

- Log in as Admin
- Log in as User
- Log out

1 test → 3 journeys

Admin — 4

- Audit log
- Email management
- Purchase-order columns
- Region config

4 tests

Client — 6

- RFP list & detail
- Purchase-order list
- Activity-report list
- Draft RFP create + delete
- Proposals
- Partner domain

6 tests

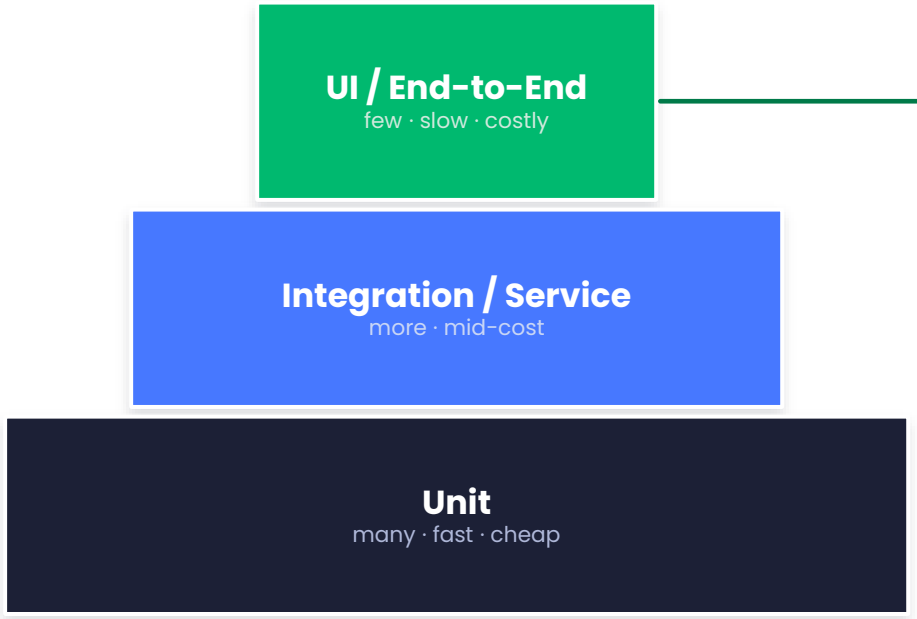
Supplier — 7

- Marketplace
- Applications
- Purchase orders
- Activity reports
- Invoices
- Browse missions · FL
- Skills · FL

7 tests · incl. 2 Freelancer

Freelancer flows live inside Supplier under a test-name prefix.

The Test Automation Pyramid — and Where Smoke Sits

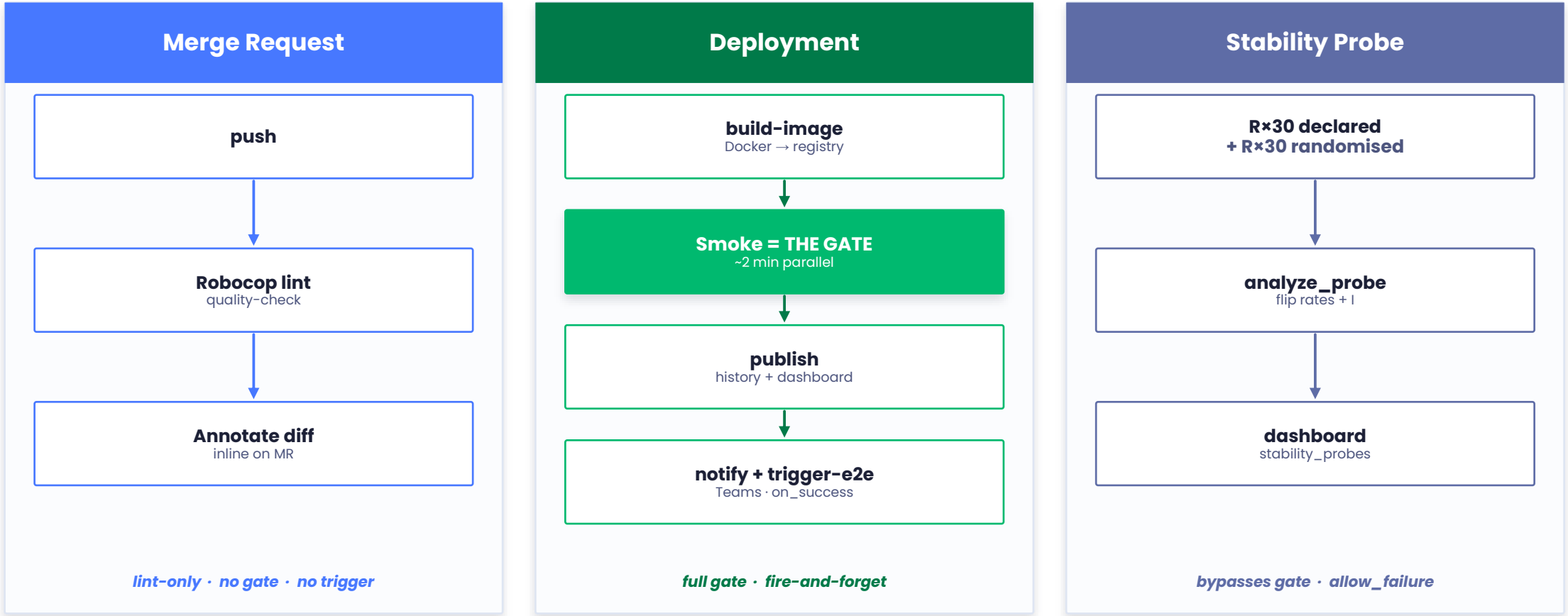


Smoke suite = minimal top slice
critical paths only — honours the pyramid

The full regression suite
is the broad top layer — ≈1,000 checks, run after the gate. My smoke suite is the minimal one in front of it.

A smoke suite is the smallest responsible slice of the top layer: it carries end-to-end cost, taken only on the critical paths.

Three Pipeline Shapes — Gate Matched to the Trigger



Each trigger has its own risk profile, so each gets its own flow: a lint-only check, the full gated deployment run, and a measurement-only probe.

Reliability Metrics — Formulas

Flakiness rate φ

$$\varphi = \frac{1}{N} \sum_{t=1}^{N} \frac{f_t}{R-1}$$

f_t = pass-to-fail transitions for test t over R runs

R - 1 = the maximum possible transitions

Target < 0.02 **Measured φ = 0.77%**

Test independence I

I = P_{random} / $P_{sequential}$

ratio of randomised-order to declared-order pass rate

≥ 1.0 = order-agnostic

Measured I = 1.004 (1.000 / 0.996)

Two measurements from one experiment: the suite runs R times in declared order (flakiness) and R times randomised (independence), against a fixed build.